

M4 自動微調 Agent

可擴充的醫學影像微調編排、決策與安全設計

依 agent/ 與 docs/ 設計文件分析
2026-07-25

M4 自動微調 Agent

從資料集到最佳訓練配方的安全自治系統

- 定位：既有 M4 微調 pipeline 的「編排層 + 決策層 + 元件組合層」
- 核心輸入：資料集、預測任務、評估設定與資源預算
- 核心輸出：跨 encoder 的最佳 Recipe、可重現實驗記錄與報告

設計目標不是取代 `main_finetune.py`，而是讓已驗證的訓練核心能被安全地自動選擇、組合、反覆改良。

問題與設計原則

為何需要 Agent？

使用者需要

哪個預訓練模型適合？

要怎麼設定 head、loss、增強與超參數？

效能不佳或訓練失敗怎麼辦？

如何確保結果可審查與可重現？

Agent 的回答

多 encoder 選擇與廣度比較

結構化、可組合的 Recipe

解答樹搜尋、除錯、續訓與配方變異

Schema、ledger、provenance、搜尋樹與報告

四項原則

1. 擴充優先：新增 encoder / head / loss / augmentation 以註冊完成，不改核心。
2. 重用既有 **pipeline**：訓練與評估核心保持在 `main_finetune.py` / `engine_finetune.py`。
3. 決策可抽換：Heuristic、LLM 與 Skill 共用 Advisor 介面。
4. 契約優先：模組與決策輸出均由 Pydantic schema 驗證。

系統全貌

從資料分析到最佳配方

- 資料平面負責檔案、影像、訓練與評估。

- 控制平面只做決策；LLM 僅能看到消毒後、受約束的 facts。

核心抽象：**Recipe** 是搜尋單位

將「一次訓練」轉成可驗證、可變異的配方

Recipe = Encoder ⊕ Head(s) ⊕ Pooling ⊕ Regularizers ⊕ Augmentation ⊕ Losses ⊕ Hyperparameters

類別	現有實作	擴充方向
Encoder	ViT 與已註冊預訓練權重	非 ViT backbone
Head	linear、MLP	segmentation、regression、multi-head
Loss	cross entropy、weighted CE、focal	Dice、MSE、輔助損失
Regularizer	drop path、weight decay、mixup、cutmix	EMA、R-drop、stochastic depth
Evaluation	EvalConfig、自訂 metric、多 fold mean±std	任務專屬指標組

關鍵元件

- **ComponentRegistry**：能力白名單與相容性基礎。
- **RecipeBuilder**：把 Recipe 映射成 main_finetune.py CLI；擋下不相容組合。
- **provenance**：記錄配方來源、父 trial、mutation、GPU 最佳化，形成可追溯譜系。

決策層：可抽換且受限制

Advisor 只提出決策，不直接訓練或產碼

實作	適用情境	行為
HeuristicAdvisor	預設與離線退路	規則式 encoder 選擇、Recipe 起點與變異階梯
LLMAdvisor	複雜決策	Claude structured output；輸出必須符合 schema
SkillAdvisor	未來專用決策能力	檔案式 handoff；介面與其他 Advisor 相同

Advisor 的責任

1. select_encoders()：提出多個 encoder / adaptation 候選。
2. compose_recipe()：從 TaskTemplate 與 registry 組出可執行配方。
3. plan_information()：只可點名核可分析器，或提出結構化問題請使用者回答。
4. review_and_decide() / propose_debug()：依歷史與安全錯誤事實選擇變異、除錯或停止。

5. `review_search_choice()`：可覆核 policy 的節點選擇，但 `LoopController` 會驗證合法性。

LLM 的自由度被限制在「有限且可驗證的決策空間」；實體化與執行永遠由純程式負責。

搜尋與實驗編排

LoopController 的單一全域解答樹

每個 trial 是一個節點；樹跨 encoder 維護全域最佳，而非各自孤立迴圈。

階段	觸發與目的
draft	起手開滿多個起點，輪替 encoder 與 preset，取得廣度
debug	選擇失敗 leaf；從 ErrorFacts 做最小修正（例：OOM→減 batch）
resume	曲線未收斂且具 checkpoint 時，受護欄控制地續訓
improve	對成功節點依 primary score softmax 抽樣，做正交 Recipe 變異

停止與資源護欄

- `max_trials` 硬上限、wall-clock 預算、使用者中斷、Advisor 停止決策。
- 完成 `min_trials` 後，連續 `patience` 次 `improve` / `resume` 無顯著提升即停止。
- 多 fold 採 **screen-then-expand**：先單 fold 篩最佳配方，再擴展至 sibling folds 彙整。

訓練、評估與可重現性

保持核心穩定，將自動化包在外圍

執行路徑

Recipe → RecipeBuilder → CLI → Trainer subprocess → `main_finetune.py` → Evaluator

- **Trainer**：隔離 subprocess、實際 CUDA 配置判斷 GPU、監測 GPU 利用率與記憶體。
- **Evaluator**：讀取 test metrics；`EvalConfig` 指定 primary metric、報告 metric 與多 fold 聚合。
- **GPU 自適應最佳化**：利用率低時，在後續 trial 調整 batch size 與 dataloader workers；高解析度資料可啟用預縮圖快取。

- **Code workspace**：若允許編輯，變更只套用在 <run_dir>/src/ 副本，原始訓練程式保持不動。

落地產物

```
runs/<experiment>/
├─ config.yaml / dataset_profile.json
├─ ledger.jsonl                # 每個 TrialResult 與 Recipe
├─ search_tree.json           # 全域搜尋譜系
├─ trials/<trial>/ checkpoint、metrics、圖表
├─ privacy_audit.jsonl        # 外送稽核
└─ report.md                  # 每 encoder 最佳與全域最佳
```

資料圍欄：把隱私視為架構不變式

核心規則：LLM 永遠看不到使用者輸入資料

兩條合法資訊管道

1. 管道 **A**：核可分析器 — LLM 只能點名 registry 中的分析器；輸出為數值、布林或列舉，不含自由字串。
2. 管道 **B**：使用者回答 — 透過 UI 結構化表單取得 modality、anatomy 等人類才知道的資訊。

隱私防線與可稽核性

Fail-closed，而非「盡量過濾」

防線	機制	解決的問題
安全事實契約	DatasetProfile 降階為 Dataset Facts；路徑移除、類別假名化	中繼資料識別洩漏
單一出口	privacy/egress.py 先 guard、再 audit、最後 API 呼叫	未審核 prompt 外送
執行期 sentinel	偵測未經 egress 的 Anthropic 呼叫並拋錯	新增繞道或直接 API 呼叫
結構化錯誤回饋 strict 模式	raw log → ErrorFacts 關閉 code edit、structured log feedback	log 含資料路徑或樣本名稱 LLM 改程式後經 log 主動讀取資料
稽核與測試	privacy_audit、canary leak、sentinel、schema、guard tests	可檢查、可驗收的保證

安全不是只靠 prompt 規範：它由 allowlist schema、執行期攔截、設定巨集與測試共同保證。

Web 工作台與操作體驗

使用者保留控制權

- 工作台：顯示決策討論、即時 trial 狀態、GPU 警示與全域解答樹。
- 實驗管理：新實驗、資料集選擇 / 安全上傳 / 格式驗證、預設超參起點、背景工作與歷史 runs。
- 互動資訊蒐集：將 LLM 問題渲染成表單卡；blocking 問題會等待，逾時採保守預設。
- 可解釋的搜尋：顯示本輪從哪個節點出發、選中機率、是否被決策層覆寫、配方 mutation 與全域最佳。
- 隱私可見性：工作台提供 audit 摘要；完整設計規劃獨立 privacy 檢視能力與本機 alias 對照。

典型操作

1. 提供 ImageFolder 資料集與評估設定。
2. Agent 分析結構、建立安全 facts，選擇 encoder 起點。
3. 依預算執行解答樹搜尋；使用者可留言引導或中止。
4. 取得可重跑的 config、最佳 Recipe、分數與完整實驗脈絡。

現況、邊界與下一步

已落地能力與誠實限制

已完成的主軸

- P0-P7：骨架、多 encoder、LLM 決策、Recipe 元件化、改良迴圈、自訂評估、TaskTemplate、Skill 接口。
- 隱私 P0-P2、P5：圍欄、分析器、安全錯誤回饋、使用者問答與 Skill handoff 對齊。
- 現有分類流程已具 linear / MLP head、weighted CE / focal loss、mixup 等元件化支援。

仍在擴充中的邊界

- 非分類 head (segmentation / regression)、多任務多 head、部分預留元件尚未完全接通訓練核心。
- Privacy P3 (完整可證明性 UI) 與 P4 (standard 模式 AST 檢查、類別逐一授權) 尚待完成。
- SkillAdvisor 的實際 Claude Code skill 呼叫仍可再接實作。

結論

M4 Agent 將模型選擇、配方工程、實驗搜尋與隱私治理整合成一個可替換、可驗證、可重現的自動微調系統；訓練核心保持穩定，智能與安全則由外圍架構持續演進。